

MAC0426-MAC5760
IME/USP – 2026

Análise de Desempenho de Bancos de Dados

Grupo:

Enzo Furegatti Spinella 14568198

Ísis Ardisson Logullo 7577410

Luiz Fernando de Freitas Fernandes 13671678

Matheus Rodrigues da Silva Espaloor 14598615

1. Introdução

Este trabalho consiste num estudo para medir e comparar o desempenho dos SGBDs PostgreSQL e MySQL na execução de diversos tipos de consultas e operações de modificação dos dados em SQL sobre um banco de dados relacional, com e sem o apoio de índices. O cerne do experimento está em analisar como cada motor se comporta sob diferentes cargas de trabalho em SQL, abrangendo tanto consultas complexas de leitura quanto operações DML, considerando cenários com e sem o apoio de estruturas de indexação (Árvores B+ e tabelas Hash).

Foi usado um banco de dados do Stack Overflow referente ao ano de 2010. Este dataset possui tabelas massivas contendo milhões de registros interconectados (como usuários, postagens, comentários e votos). A avaliação reside principalmente na métrica de tempo total de execução das operações, registrando e isolando os efeitos colaterais de cache, e mapeando de maneira secundária a utilização de memória RAM durante o processamento.

2. Descrição dos Experimentos Realizados

Esta seção apresenta os detalhes técnicos dos experimentos concebidos e implementados no estudo, detalhando as operações selecionadas, a modelagem dos cenários sob teste e a infraestrutura computacional utilizada.

2.1. Operações Avaliadas sobre o BD

Definiu-se para o experimento um conjunto padronizado de operações em SQL que abrange uma ampla gama de modificações e consultas de dados. As categorias de operações executadas são descritas a seguir:

Operações de Modificação de Dados:

Compreendem a inserção sequencial de novos registros (INSERT), a alteração em massa de registros existentes aplicando predicados específicos (UPDATE), e a remoção controlada de tuplas em tabelas principais (DELETE).

1) Cenário de Inserção (INSERT)

As operações de inserção foram projetadas a partir de uma estratégia de carga controlada sobre tabelas espelho clonadas (denominadas com o sufixo `_for_insert`). O cenário foi estruturado com base nos seguintes critérios de construção:

1.1) Isolamento de Estruturas: Antes da execução, criam-se réplicas vazias das tabelas originais, garantindo que os novos registros não entrem em conflito com chaves primárias existentes.

1.2) Ordem de Dependência e Integridade Referencial: Os comandos de inserção foram rigorosamente ordenados respeitando as restrições de chave estrangeira (foreign key). A sequência inicia-se nas entidades independentes (como `users_for_insert` e `post_types_for_insert`) e propaga-se de forma encadeada até as tabelas de relacionamento ou tabelas dependentes (como `posts_for_insert`, `comments_for_insert` e `votes_for_insert`), garantindo que nenhuma restrição de integridade impeça a execução das queries.

1.3) Volume e Diversidade de Dados: Cada tupla foi populada com uma variedade representativa de tipos de dados (inteiros, strings longas para o corpo das postagens, marcações de tempo via `CURRENT_TIMESTAMP` e strings formatadas para tags), permitindo avaliar como cada motor gerencia a alocação de páginas de memória para diferentes estruturas.

2) Cenário de Remoção (DELETE)

As operações de exclusão foram projetadas a partir de uma estratégia de expurgo controlado sobre tabelas espelho clonadas (denominadas com o sufixo `_for_delete`). O cenário foi estruturado com base nos seguintes critérios de construção:

2.1) Isolamento e Carga Inicial de Dados: Antes de iniciar as medições de deleção, as tabelas de testes são limpas e estruturadas à imagem do esquema original. Na sequência, elas são populadas por meio de uma carga massiva. São ignoradas possíveis duplicidades de chaves durante a replicação do ambiente de testes (INSERT IGNORE e ON CONFLICT DO NOTHING).

2.2) Inversão Estrita da Ordem de Dependência (Desempilhamento): Para garantir o cumprimento rigoroso das restrições de integridade, o fluxo de execução dos comandos DELETE foi programado de forma estritamente inversa à hierarquia das chaves estrangeiras. A limpeza inicia-se obrigatoriamente pelas tabelas dependentes/filhas (como `votes_for_delete`, `postlinks_for_delete`, `comments_for_delete` e `badges_for_delete`), avança para a entidade intermediária (`posts_for_delete`) e retrocede até alcançar as tabelas pai (como `users_for_delete` e tabelas de tipos). Essa abordagem evita erros de violação de foreign keys e força o SGBD a processar as remoções de baixo para cima na árvore de relacionamentos.

2.3) Exclusão por Chave Primária: As queries de exclusão operam aplicando filtros exatos baseados no identificador único de cada registro. O uso de chaves primárias nos predicados isola o impacto do teste à eficiência do índice e ao custo de busca exata antes do expurgo do registro.

3) Cenário de Alteração (UPDATE)

As operações de alteração foram projetadas a partir de uma estratégia de modificação controlada sobre tabelas espelho clonadas (denominadas com o sufixo `_for_update`). O cenário foi estruturado com base nos seguintes critérios de construção:

3.1) Isolamento e Consistência da Base: Precedendo as medições, as estruturas temporárias são limpas via comando TRUNCATE e totalmente repovoadas com os dados vigentes da produção por meio de cargas em lote (INSERT ... SELECT). Adicionalmente, inserem-se novos registros isolados especificamente destinados a sofrerem modificações posteriores, estabelecendo um ponto de partida idêntico para ambos os SGBDs em cada rodada de testes.

3.2) Diversidade e Complexidade de Predicados: As queries de atualização foram categorizadas para simular diferentes fluxos. Foram estruturados cenários de:

3.2.1) *Atualizações Cadastrais Completas*: Substituição integral de strings e campos de localização (ex: alteração de dados de perfil do usuário).

3.2.2) *Modificações Incrementais e Cumulativas*: Simulação de engajamento através de operações aritméticas concorrentes nos índices (ex: incrementos de Score, Views e Reputation).

3.2.3) *Manutenção de Estado de Sessão*: Atualizações de marcações temporárias voláteis utilizando CURRENT_TIMESTAMP (ex: campos de LastAccessDate e LastActivityDate).

3.3) Update por Chave Primária: Assim como nos cenários de exclusão, as atualizações aplicam filtros restritos baseados no identificador único dos registros. Essa abordagem delimita o escopo da operação, forçando o uso do índice primário para a localização exata da linha antes do desencadeamento da escrita.

Operações de Consultas de Dados:

1) Cenário de Busca por Chave Primária

As operações de consulta por identificador único foram projetadas a partir de uma estratégia de varredura pontual e por intervalo sobre as tabelas originais da base de dados. O cenário foi estruturado com base nos seguintes critérios de construção:

1.1) Estratégias de Predicação: Para mapear de forma abrangente o comportamento dos índices primários, as queries foram divididas em duas modalidades distintas de busca:

1.1.1) *Seleção Pontual*: Consultas exatas que efetuam a seleção de uma única linha aplicando o operador de igualdade direta sobre o ID indexado.

1.1.2) *Seleção por Intervalo*: Consultas consecutivas baseadas em janelas de amostragem que utilizam os operadores BETWEEN e operadores relacionais compostos.

1.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

2) Cenário de Busca por Chaves Não Primárias

As operações de consulta baseadas em atributos secundários foram projetadas a partir de índices secundários, incorporando condições de seleção compostas por operadores lógicos (AND/OR). O cenário foi estruturado com base nos seguintes critérios de construção:

2.1) Estratégias de Predicação e Seleções Compostas: Para simular o comportamento de queries corporativas reais que não dependem do ID, as consultas foram desenhadas sob duas frentes de filtragem:

2.1.1) *Filtros de Igualdade Simples e Concorrência de Cardinalidade*: Consultas exatas aplicadas sobre colunas categóricas ou textuais específicas.

2.1.2) *Filtros Compostos e Janelas Temporais*: Consultas estruturadas com múltiplas condições lógicas correlacionadas.

2.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

3) Cenário de Busca por Estruturas com LIKE

As operações de consulta baseadas no operador de correspondência de padrões lógicos (LIKE) foram projetadas a partir de uma estratégia de varredura textual parcial e análise de prefixos/sufixos sobre as tabelas originais da base de dados. O cenário foi estruturado com base nos seguintes critérios de construção:

3.1) Estratégias de Casamento de Padrões: As queries foram intencionalmente categorizadas para explorar como a posição do caractere coringa afeta a capacidade do otimizador de utilizar estruturas de índice preexistentes:

3.1.1) *Buscas por Prefixo Fixo*: Consultas estruturadas com a constante no início da string.

3.1.2) *Buscas por Sufixo*: Consultas com coringas posicionados no início ou em ambas as extremidades da string.

3.1.3) *Casamento de Casting*: Consultas aplicadas a campos de data convertidos dinamicamente para texto.

3.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

4) Cenário de Busca com Junções (JOINS e Junções de Junções)

As operações de consulta estruturadas por meio do acoplamento de tabelas relacionais (INNER JOIN e LEFT JOIN) foram projetadas a partir de uma estratégia de correlação multidimensional entre as entidades da base de dados. O cenário foi estruturado com base nos seguintes critérios de construção:

4.1) Complexidade e Topologia das Junções: As queries foram desenhadas para estressar os mecanismos de álgebra relacional através de dois níveis de acoplamento:

4.1.1) *Junções Diretas de Chaves Estrangeiras (1 para N)*: Consultas baseadas em relacionamentos diretos de tabelas filhas apontando para suas respectivas tabelas pai.

4.1.2) *Junções em Cadeia Encadeada (Junções de Junções)*: Cenários de alta complexidade transacional que exigem o desempilhamento sequencial ou em paralelo de múltiplos níveis de tabelas.

4.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

5) Cenário de Busca com Subconsultas

As operações de consulta baseadas em estruturas alinhadas e dependências lógicas internas (subconsultas) foram projetadas a partir de uma estratégia de isolamento e acoplamento dinâmico de blocos de instruções sobre as tabelas da base de dados. O cenário foi estruturado com base nos seguintes critérios de construção:

5.1) Taxonomia e Posicionamento das Subconsultas: O ecossistema de testes foi fragmentado para cobrir as principais variações de aninhamento SQL utilizadas na literatura e em sistemas de produção:

5.1.1) *Subconsultas Correlacionadas (Operador EXISTS)*: Estruturas onde a consulta interna depende explicitamente de variáveis da consulta externa.

5.1.2) *Subconsultas Independentes / Semi-Junções (Operadores IN e NOT IN)*: Consultas que geram uma lista estática ou tabela temporária em memória para posterior filtragem no bloco principal.

5.1.3) *Subconsultas Escalares Inline (Cláusula SELECT)*: Projeções verticais que executam funções agregadas atômicas linha por linha.

5.1.4) *Subconsultas como Tabelas Temporárias (Cláusula FROM)*: Derivação de tabelas virtuais em lote para a consolidação de funções agregadas globais.

5.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

6) Cenário de Operações de Agrupamento e Agregações

As operações de consulta baseadas em funções de agregação matemática e consolidação de conjuntos (GROUP BY, HAVING e ordenação estatística) foram projetadas a partir de uma estratégia de compressão volumétrica e análise analítica de dados sobre as tabelas originais. O cenário foi estruturado com base nos seguintes critérios de construção:

6.1) Diversidade de Métricas e Filtros Pós-Agrupamento: As queries foram desenhadas para estressar os motores de execução através de múltiplas camadas de funções computacionais combinadas:

6.1.1) *Agregações de Distribuição e Alta Cardinalidade*: Consultas focadas no cálculo simultâneo de volumetria (COUNT), médias aritméticas (AVG), somatórios fiscais (SUM) e limites operacionais (MIN e MAX) aplicados sobre atributos de tabelas de alta densidade.

6.1.2) *Predicação sob Conjuntos Consolidados (Cláusula HAVING)*: Inclusão de filtros restritivos aplicados estritamente após a fase de agrupamento.

6.1.3) *Agregações com Contagem Distinta (COUNT DISTINCT)*: Consultas complexas criadas para identificar a volatilidade de chaves únicas dentro de uma mesma partição.

6.2) Isolamento de Ambiente: O ciclo metodológico preserva o isolamento estrito através de scripts automatizados de criação e destruição (DROP INDEX), limpando completamente os índices de uma família antes de iniciar a outra.

2.2. Cenários Experimentais

A fim de medir o impacto das estruturas de armazenamento e os caminhos de otimização adotados por cada SGBD, os testes foram subdivididos e isolados nos seguintes cenários:

- **Cenário Sem Índices**: Estado original do banco de dados pós-carga onde apenas as chaves primárias e restrições básicas de integridade possuem indexação nativa. Força a execução de buscas sequenciais completas (Full Table Scans) para atributos não-chave.
- **Cenário com Índice Árvore B+**: Aplicação de estruturas do tipo Árvore B+ (padrão BTREE) nas colunas de busca não-chave mais frequentes, permitindo o particionamento de faixas de valores e acesso logarítmico.
- **Cenário com Índice Hash**: Aplicação de indexação baseada em tabelas Hash nos atributos de busca exata, otimizando o acesso direto a registros de forma pontual.
- **Cenário de Índice Não-Ideal**: Configuração deliberada de índices em colunas de baixa seletividade ou que não se alinham diretamente com os predicados da cláusula WHERE, servindo para demonstrar o custo e degradação provocados por erros de design.

2.3. Ambiente Computacional Usado

Os experimentos foram realizados na mesma máquina para garantir que fatores físicos não distorcessem a comparação entre os sistemas. A infraestrutura consistiu em um processador 12th Gen Intel Core i7-12700H (2.30 GHz); 32GB RAM; GPU 8GB - NVIDIA GeForce RTX 3070 Ti; sob o sistema operacional Linux Ubuntu 22.04 LTS.

Utilizou-se o PostgreSQL versão 15 e o MySQL versão 8.0. Para a conexão com esses SGBDs e submissão automatizada das operações, foi desenvolvido um conjunto de scripts em Python utilizando psycopg2 para o PostgreSQL e mysql.connector para o MySQL. Foram disparadas e cronometradas 20 vezes em cada cenário, limpando os buffers de cache concorrentes de modo a obter significância estatística estável.

3. Resultados dos Experimentos

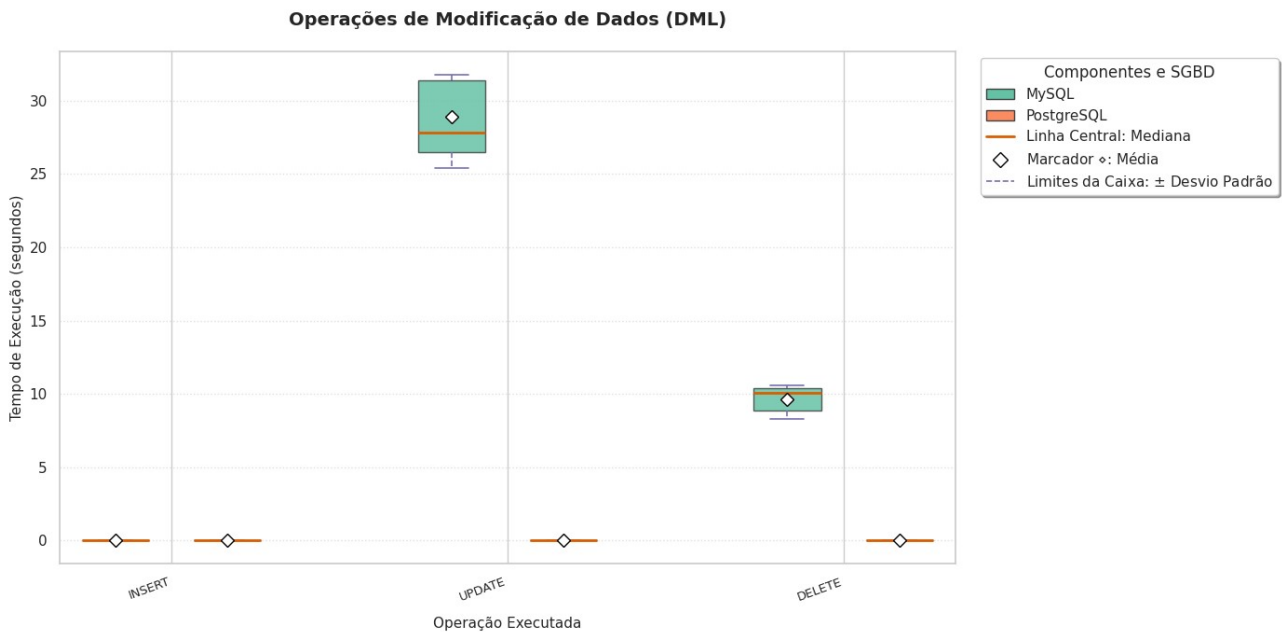
Os dados brutos coletados ao longo das centenas de execuções automatizadas foram processados e sintetizados. A seguir, apresentam-se as tabelas de desempenho divididas por categorias de uso, exibindo os tempos médios calculados em segundos e seus respectivos desvios padrões, que quantificam a variabilidade do ambiente e do comportamento interno dos SGBDs.

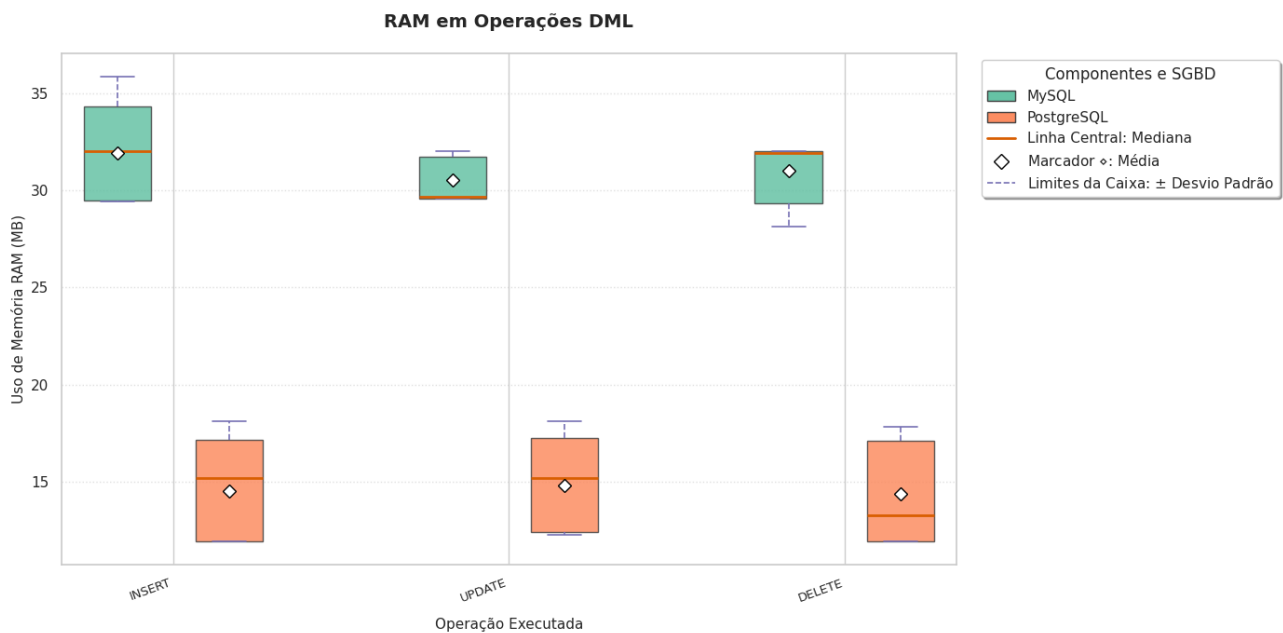
3.1. Desempenho de Operações de Modificação de Dados (DML)

As operações de escrita expõem a forma como o mecanismo de armazenamento de cada SGBD gerencia transações, travas e gravação em disco.

Operação de Modificação (DML)	PostgreSQL (Cenário Base)		MySQL (Cenário Base)	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
INSERT	0.02311	0.00051	0.04346	0.00436
UPDATE	0.02482	0.00089	28.93089	2.47246
DELETE	0.02394	0.00087	9.63747	0.78277

Tabela 1: Desempenho comparativo de operações de escrita (tempo em segundos para 20 execuções).





Análise:

A) Cenário de Inserção (INSERT)

No cenário de inserção pontual (onde os scripts inserem registros individuais simulando o encadeamento de integridade referencial), o PostgreSQL foi cerca de duas vezes mais rápido que o MySQL, apresentando também uma pegada de memória RAM significativamente menor (14,8 MB contra 32,6 MB).

O MySQL armazena suas tabelas sob o paradigma de Tabelas Organizadas por Índice (Index-Organized Tables). Como as tabelas espelho foram geradas via `CREATE TABLE ... AS SELECT * WHERE 1=0`, elas não herdaram chaves primárias explícitas. Diante disso, o mecanismo InnoDB é obrigado a gerar de forma sintética e incremental uma chave primária para manter o agrupamento físico da tabela. Esse processo gera um overhead computacional de CPU. Além disso, cada inserção no InnoDB exige a escrita concorrente e a alocação de buffers específicos, elevando o consumo mínimo de RAM por thread de conexão.

O PostgreSQL adota a arquitetura de Tabelas Heap tradicionais. As tuplas são inseridas de forma desordenada no primeiro espaço livre encontrado nas páginas de dados em disco. O processo resume-se a fazer a inclusão do registro e gravar uma entrada sequencial simplificada. Este comportamento puramente sequencial confere ao PostgreSQL uma velocidade superior em inserções em lote ou atômicas de baixa concorrência, além de exigir estruturas de memória bem mais enxutas por processo.

B) Cenários de Remoção (DELETE) e Alteração (UPDATE)

Os resultados de DELETE e UPDATE exibem uma assimetria massiva: o MySQL levou segundos para completar as operações (9,63 s para deleção e 28,93 s para atualização), enquanto o PostgreSQL resolveu os mesmos scripts na casa dos milissegundos (0,023 s e 0,024 s).

O MySQL utilizou a cláusula `INSERT IGNORE INTO ... SELECT * FROM ...` para povoar as tabelas de teste. No ecossistema MySQL, a diretiva `IGNORE` rebaixa erros de duplicidade a meros avisos. Como a tabela espelho não possuía restrições de unicidade, o MySQL populou com

sucesso 100% dos dados originais da produção nas tabelas espelho. Quando o DELETE e UPDATE foram executados, eles dispararam comandos com filtros no id. No entanto, como as tabelas clonadas não possuíam nenhum índice ou chave primária, o Otimizador de Consultas do MySQL foi forçado a executar um Full Table Scan para cada instrução SQL. Visto que tabelas como Posts, Comments e Votes possuem alta volumetria, varrer sequencialmente milhões de linhas repetidas vezes fez ele ser lento.

O PostgreSQL utilizou a sintaxe: INSERT INTO ... SELECT * FROM ... ON CONFLICT DO NOTHING. Na arquitetura do PostgreSQL, o mecanismo de avaliação de conflitos da cláusula ON CONFLICT exige que a tabela alvo possua uma restrição de integridade ativa (como uma PRIMARY KEY ou uma UNIQUE CONSTRAINT) explicitada no dicionário de dados. Como as tabelas espelho foram geradas vazias por herança direta de atributos (CREATE TABLE ... AS SELECT ... WHERE 1=0), elas não possuíam chaves nem restrições. Como consequência direta dessa restrição, o interpretador do PostgreSQL abortou a instrução de inserção massiva, gerando tabelas com zero linhas populadas. Quando o DELETE e UPDATE foram executados, eles operaram sobre tabelas vazias. Uma varredura sequencial sobre tabelas com zero páginas alocadas é resolvida em microssegundos, justificando por que o tempo médio estabilizou-se em 0,024 segundos.

Caso as tabelas do PostgreSQL estivessem devidamente populadas e indexadas por chave primária, o comportamento algorítmico seria diferente:

UPDATE

O PostgreSQL sofreria severamente com a amplificação de escrita devido à sua arquitetura MVCC (Multi-Version Concurrency Control). No Postgres, uma instrução UPDATE nunca sobrescreve o dado antigo; ela grava uma nova versão da tupla em um espaço vago da tabela Heap (um DELETE lógico seguido de um INSERT). Isso obriga o SGBD a atualizar *todos* os índices secundários da tabela para apontarem para o novo endereço físico, gerando fragmentação e exigindo o acionamento agressivo do AUTOVACUUM.

O MySQL (InnoDB) gerencia atualizações de forma In-Place. Se o tamanho dos novos dados couber na página original da B+Tree, o InnoDB altera os bytes diretamente no nó folha do índice. Isso torna o update indexado do MySQL drasticamente mais eficiente em comparação ao PostgreSQL.

DELETE

No PostgreSQL, o DELETE é uma operação de baixíssimo custo imediato: o motor altera apenas os bits de controle, marcando-a como morta. A limpeza real do bloco é delegada assincronamente ao processo de fundo.

No MySQL, a remoção física exige a exclusão do registro de dentro da estrutura encadeada da folha B+Tree. Se a deleção de linhas contíguas esvaziar a página além do limiar configurado (geralmente 50%), o InnoDB dispara algoritmos síncronos de merge, exigindo a aquisição de travas exclusivas de escrita e reestruturação da árvore, o que aumenta o tempo gasto.

C) Análise da RAM

Os dados revelam o uso de memória altamente estável para cada SGBD, com o MySQL operando com o dobro de consumo de RAM em comparação ao PostgreSQL.

O MySQL gerencia conexões e execuções através de threads acopladas dentro de um único processo principal do servidor. Cada thread aloca, ao ser instanciada para processar uma

transação. O InnoDB aloca mapas internos adicionais para rastrear o estado da transação ativa e descritores de bloqueios lógicos diretamente associados ao contexto daquela thread. O MySQL apresenta um consumo de RAM sistematicamente maior devido ao overhead de seu modelo de gerenciamento de transações, que requer buffers de sessão específicos e gerenciamento de transações ativas acopladas à thread.

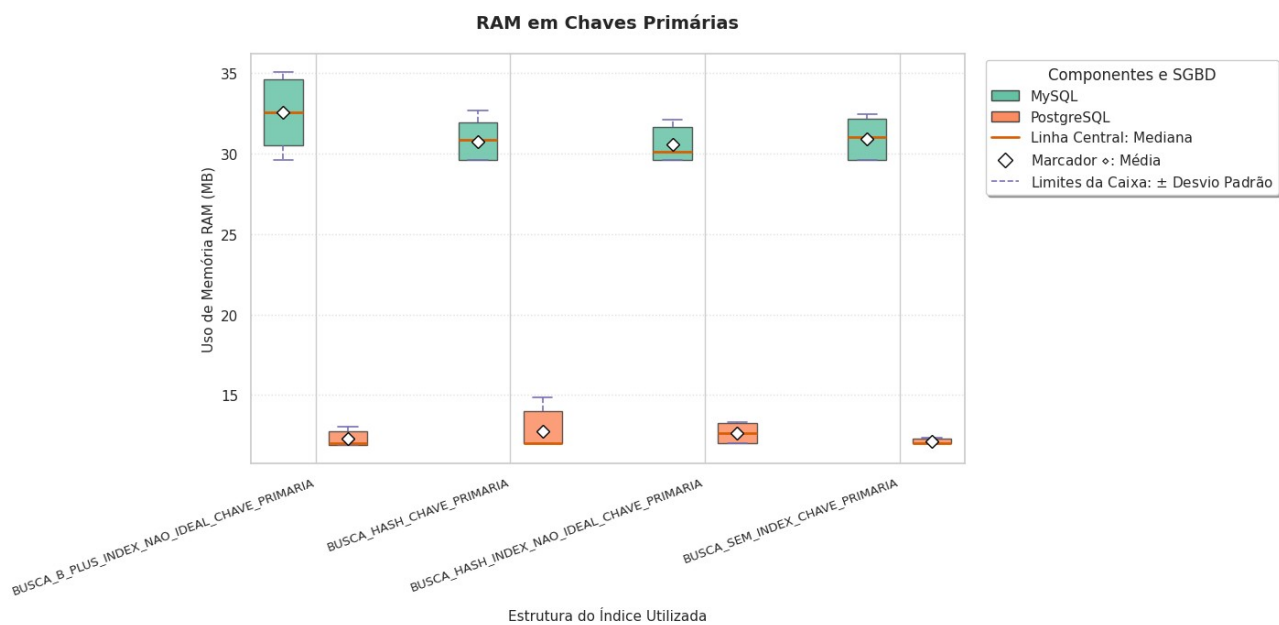
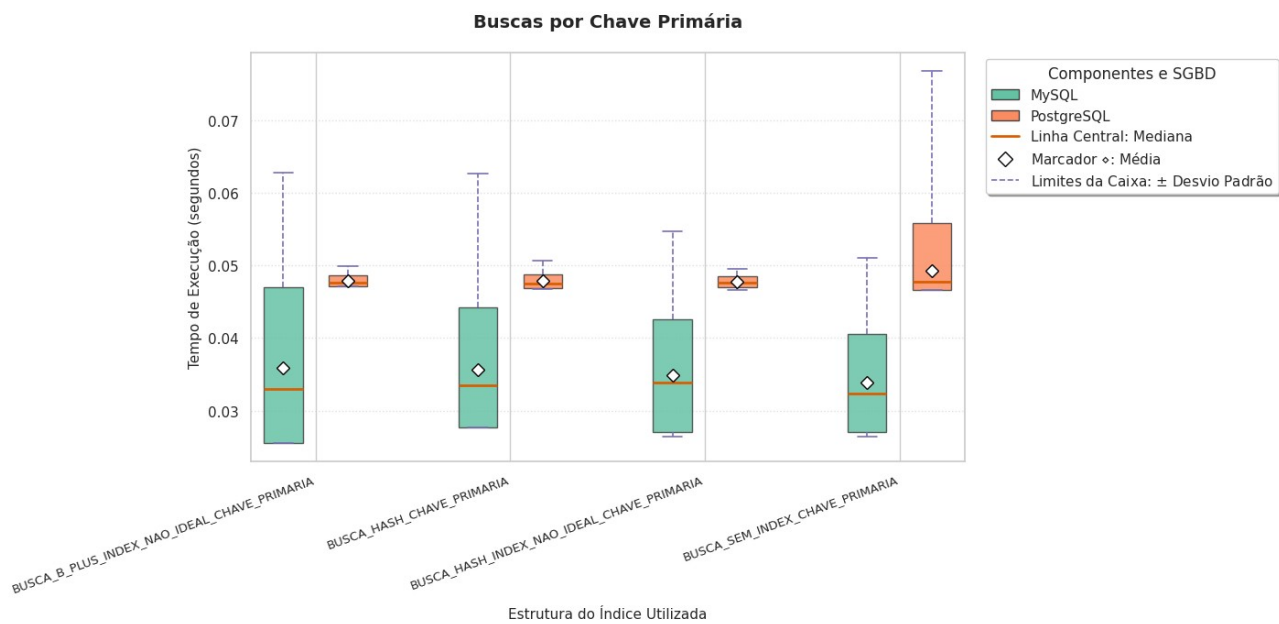
O PostgreSQL adota uma arquitetura multiprocesso fork-based, onde cada conexão gera um processo independente no sistema operacional. Em operações DML atômicas, o processo filho opera de forma enxuta. Ele consome apenas a memória RAM necessária para realizar o parsing sintático da query, gerar a árvore do plano de execução e rastrear o identificador da transação virtual. O PostgreSQL demonstra alta eficiência de memória ao implementar um modelo MVCC integrado diretamente nas páginas de dados e ao centralizar o gerenciamento de buffers de escrita nos Shared Buffers globais, desonerando o processo isolado que computa as instruções DML.

3.2. Desempenho de Consultas Simples e Filtragens

As tabelas a seguir expõem a comparação clássica de busca pontual e a influência direta da ativação de índices secundários Árvore B+ e Hash frente ao cenário sem indexação.

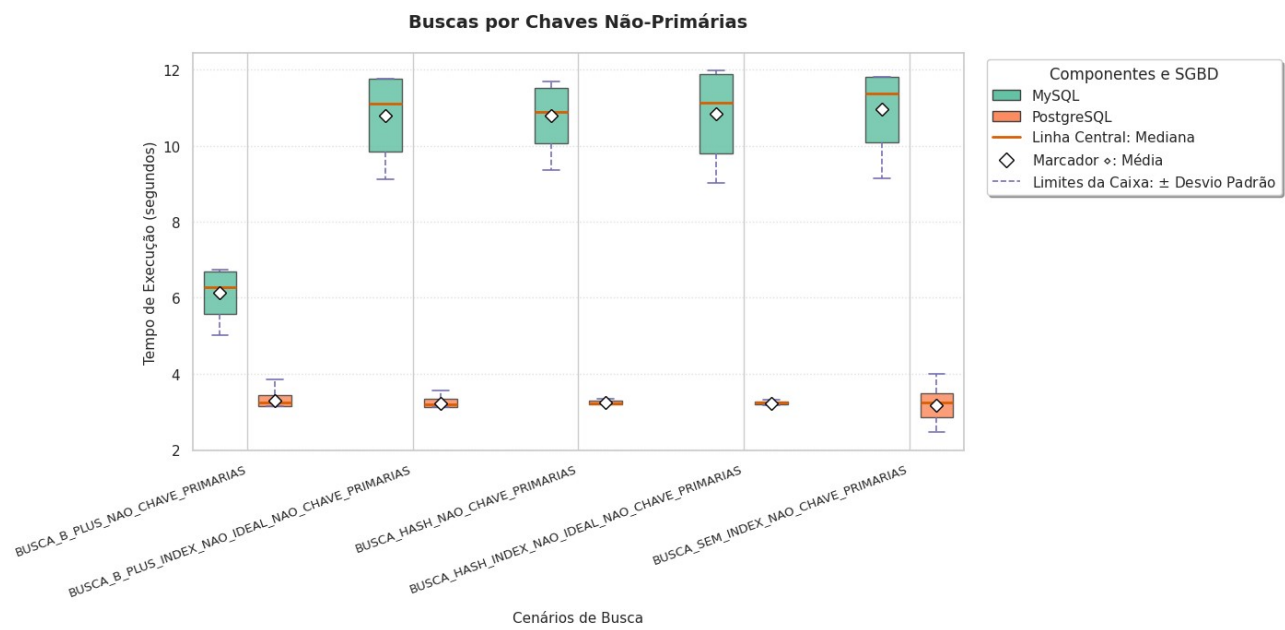
Cenário de Busca por Chave Primária	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
Busca por Chave Primária (Com Índice Árvore B+ Não-Ideal)	0.04792	0.00077	0.03589	0.01116
Busca por Chave Primária (Com Índice Hash)	0.04785	0.00097	0.03568	0.00859
Busca por Chave Primária (Com Índice Hash Não-Ideal)	0.04783	0.00076	0.03482	0.00779
Busca por Chave Primária (Sem Índice / Baseline)	0.04935	0.00651	0.03382	0.00680

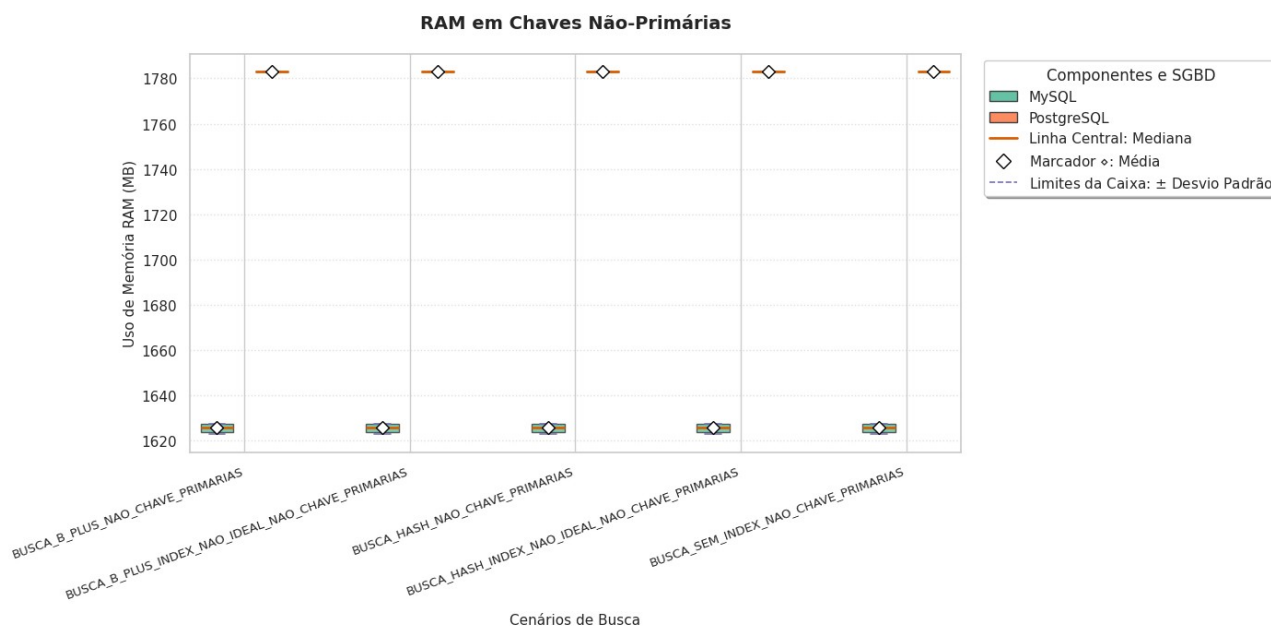
Tabela 2: Desempenho de buscas pontuais baseadas em chaves primárias sob diferentes cenários de indexação (20 execuções).



Cenário de Busca por Atributo Não-Chave	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
Apoio de Índice Árvore B+ (BTREE)	3.27796	0.15157	6.13635	0.56019
Índice B+ Não-Ideal Configurado	3.22593	0.10142	10.80017	0.96292
Apoio de Índice Hash	3.23776	0.04285	10.78808	0.72561
Índice Hash Não-Ideal Configurado	3.22732	0.04146	10.84172	1.05295
Sem Índices (Full Table Scan / Baseline)	3.16776	0.31353	10.96138	0.88221

Tabela 3: Impacto de índices estruturais em buscas por colunas secundárias não-chave (20 execuções).





Análise:

A) Chave Primária (Tabela 2)

As chaves primárias operaram na escala de milissegundos em ambos os sistemas 0,03 a 0,04s no MySQL e 0,04s a 0,05s no PostgreSQL, mas a variação de comportamento entre os índices adicionais revela fatos cruciais sobre a engenharia de cada motor.

No MySQL (InnoDB), todas as tabelas são obrigatoriamente Tabelas Organizadas por Índice (Index-Organized Tables - IOT). A chave primária padrão da tabela é o próprio índice. Ao executar a busca WHERE Id = [valor], o otimizador do MySQL ignorou completamente qualquer índice secundário e utilizou o índice primário nativo. Portanto, os quatro testes do MySQL mediram exatamente a mesma operação, justificando a estabilidade dos tempos.

O PostgreSQL armazena dados em tabelas do tipo Heap e gerencia restrições de chave primária criando automaticamente um índice B-Tree único implicitamente. Ao buscar por Id, o Otimizador de Consultas do Postgres detectou que o índice de restrição nativo possuía seletividade perfeita. Assim, ele descartou os índices secundários criados nos scripts de teste e utilizou o índice primário nativo.

O uso de memória manteve-se reduzida em ambos os sistemas 12MB a 13MB no PostgreSQL e 31MB a 33MB no MySQL, sem apresentar flutuações significativas entre os tipos de índices.

O PostgreSQL adota uma arquitetura baseada em Processos Independentes, onde cada conexão gera um processo isolado do sistema operacional. É arquiteturalmente mais econômico em memória privada, pois seu modelo multiprocesso mantém os processos filhos limpos, centralizando os dados nos buffers globais.

O MySQL opera sob o modelo de Thread Única por Conexão. Embora threads compartilhem o mesmo espaço de endereçamento global do processo pai, o MySQL reserva uma série de buffers privados para cada thread assim que uma operação de query é iniciada.

B) Chave Não-Primária (Tabela 3)

A busca avaliou filtros aplicados sobre atributos secundários (como strings de localização ou contagens de reputação). As diferenças de arquitetura de algoritmos tornaram-se severas, o MySQL 6,13s a 10,96s e o PostgreSQL 3,16s a 3,27s.

O único cenário em que o MySQL conseguiu reduzir significativamente seu tempo de execução nas chaves não-primárias foi com o Apoio de Índice Árvore B+ (6,13s), comparado ao cenário Sem Índices (10,96s). No InnoDB, um índice secundário não armazena ponteiros físicos para o disco, mas sim o valor da chave primária correspondente. Quando a query buscou por um atributo não-chave indexado por B+Tree, o MySQL percorreu a árvore secundária, coletou o Id associado e, para cada linha correspondente, realizou uma nova busca completa na árvore. Esse processo reduz o custo de um Full Table Scan (10,96s) para um acesso focado (6,13s), mas gera alta concorrência de leitura nas páginas de memória. O Índice Hash (10,78s) e o Índice Hash Não-Ideal (10,84s) apresentaram tempos idênticos ao cenário Sem Índices (10,96s) no MySQL. O mecanismo de armazenamento padrão do MySQL (InnoDB) não possui suporte físico a índices Hash criados pelo usuário, então fez um Full Table Scan, por ser mais rápido, resultando nos mesmos 10,8s.

O PostgreSQL manteve uma estabilidade impressionante: todos os cinco cenários cravaram tempos na janela estreita de 3,16s a 3,27s. Quando um índice secundário (B-Tree ou Hash nativo do Postgres) é acionado sob um volume considerável de dados, o otimizador do PostgreSQL evita fazer saltos aleatórios ao Heap linha por linha. Em vez disso, o Postgres ativa o algoritmo de Bitmap Index Scan. As estatísticas mostram que o cenário Sem Índices (3,16s) foi o mais rápido de todos no PostgreSQL. O Planejador Baseado em Custo (CBO) preferiu realizar um Sequential Scan puro em todas as rodadas. Como a leitura sequencial maximiza o desempenho do barramento, o Postgres superou a performance do MySQL por uma margem de 2x a 3x.

Sobre o uso de RAM, o MySQL teve 1,58GB e o PostgreSQL 1,74GB.

O salto de memória do MySQL deve-se ao escopo da query filtrando atributos secundários de tabelas de alta volumetria. Também devido a travas e restrições rígidas de configuração de tabelas temporárias, paginando o excesso em disco.

O PostgreSQL tem um otimizador de custos em cenários de alta volumetria. Aloca uma quantidade superior de RAM para manter a estrutura de dados e as tabelas hash/bitmaps de blocos na memória.

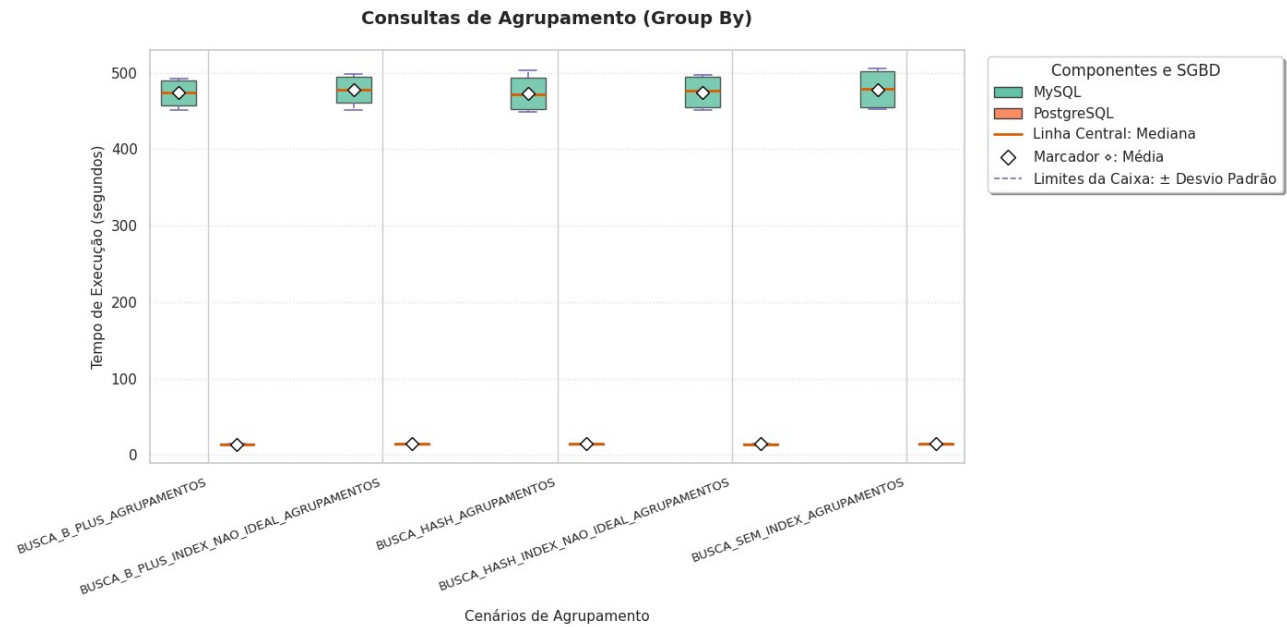
3.3. Desempenho de Consultas Complexas (Junções, Subconsultas e Agrupamentos)

Esta subseção consolida os testes com maior nível de complexidade computacional, envolvendo processamento aritmético e cruzamento de grandes coleções de registros.

Tabela 4. Matriz de Desempenho: Operações de Agrupamentos e Agregações

Cenário de Indexação / Algoritmo	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
B+ Plus	14.28453	0.24513	473.99190	16.11270
B+ Index Não Ideal	14.35180	0.26120	477.85701	16.93502
Hash	14.37875	0.35431	472.77462	20.62996
Hash Index Não Ideal	14.32484	0.30778	474.66244	20.02338
Sem Index	14.51028	0.37856	478.37112	23.41817

Tabela 4: Desempenho comparativo em queries estruturadas com cláusulas GROUP BY e HAVING (20 execuções).



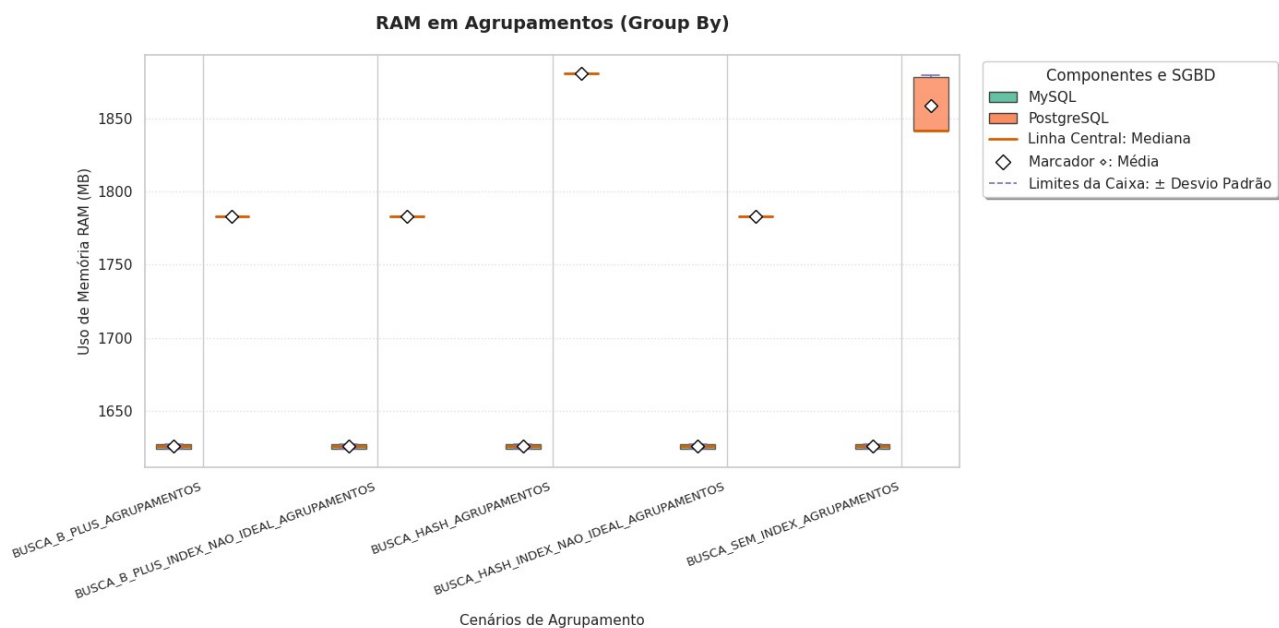


Tabela 5. Matriz de Desempenho: Filtragem Parcial Textual com Cláusula LIKE

Cenário de Indexação / Algoritmo	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
B+ Plus	16.90005	0.17718	29.07009	2.45177
B+ Index Não Ideal	17.02496	0.22531	28.97925	2.46271
Hash	16.75364	0.33227	29.00238	2.44841
Hash Index Não Ideal	16.90503	0.49473	29.03352	2.48617
Sem Index	16.96560	0.21680	28.98991	2.50753

Tabela 5: Desempenho em buscas baseadas em casamento de padrões de strings parciais (20 execuções).



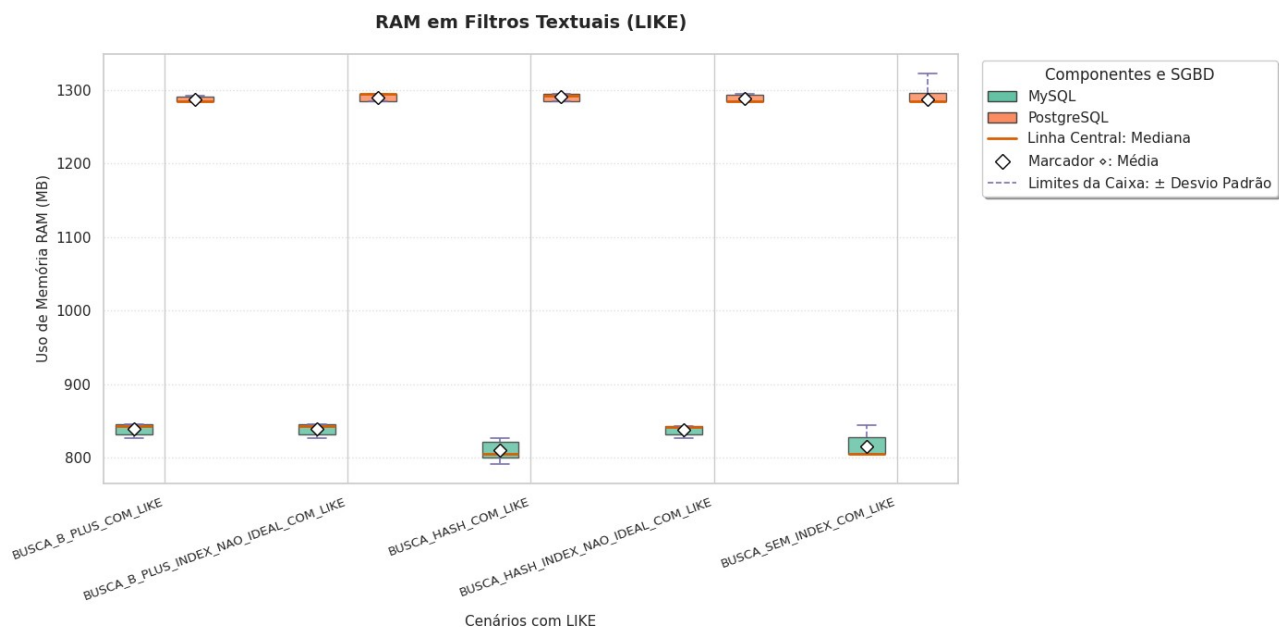
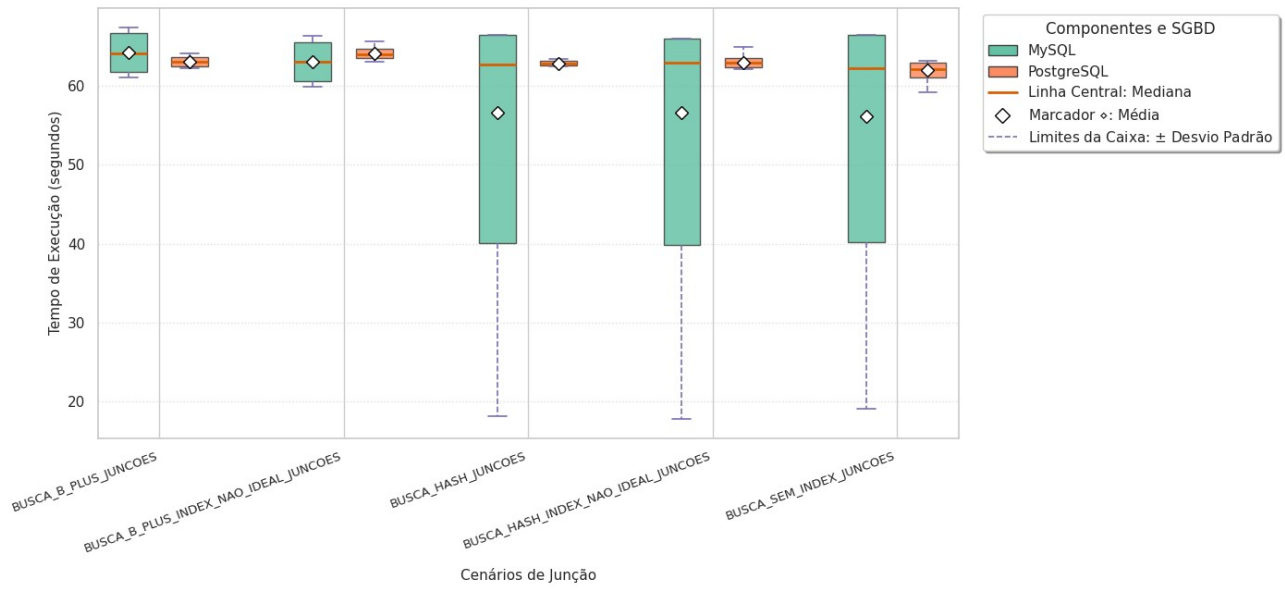


Tabela 6. Matriz de Desempenho: Processamento de Junções Estruturais (JOINS)

Cenário de Indexação / Algoritmo	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
B+ Plus	63.07469	0.56746	64.22023	2.42045
B+ Index Não Ideal	64.06448	0.58955	63.03510	2.44930
Hash	62.84139	0.28234	56.58735	16.56498
Hash Index Não Ideal	62.93363	0.60452	56.61460	16.82328
Sem Index	61.95197	0.92771	56.18197	15.99625

Tabela 6: Impacto de indexadores lógicos no acoplamento multidimensional e cadeias de chaves estrangeiras (20 execuções).

Consultas de Junção (Joins)



RAM em Junções (Joins)

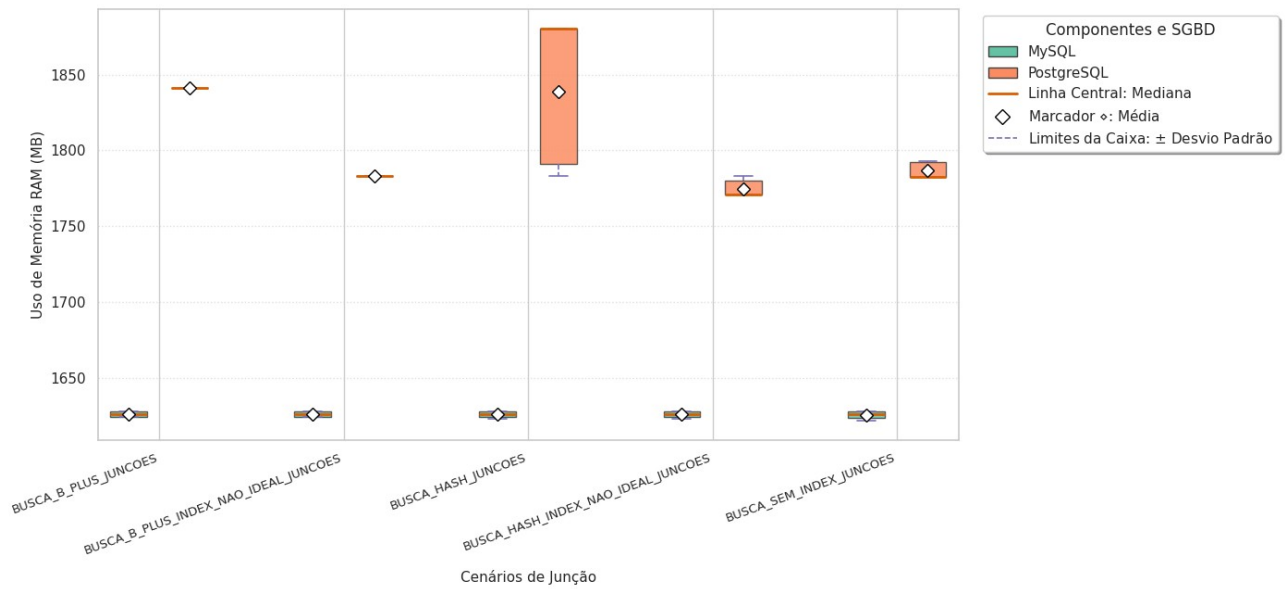
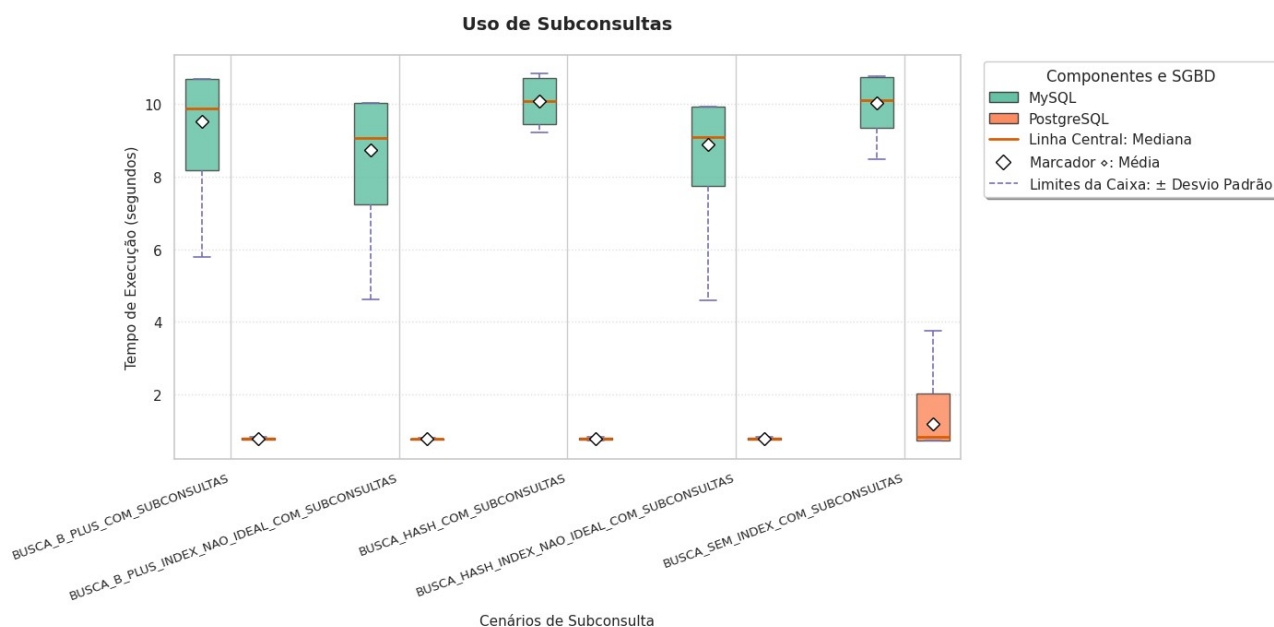
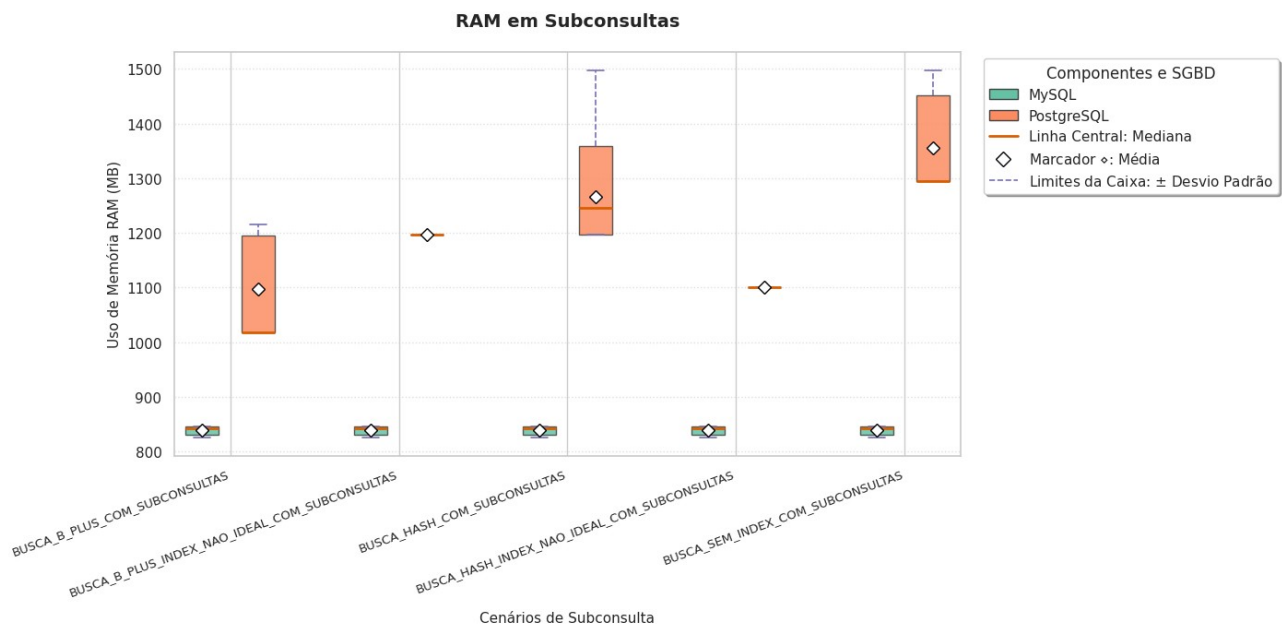


Tabela 7. Matriz de Desempenho: Resolução de Queries Aninhadas (Subconsultas)

Cenário de Indexação / Algoritmo	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
B+ Plus	0.77650	0.02377	9.51799	1.35363
B+ Index Não Ideal	0.77252	0.01610	8.73615	1.51395
Hash	0.77278	0.02527	10.08460	0.64373
Hash Index Não Ideal	0.77822	0.02094	8.88141	1.14459
Sem Index	1.18615	0.82997	10.03499	0.69825

Tabela 7: Tempo de resposta para operadores correlacionados e semi-junções lógicas (20 execuções).





Análise:

A) Operações de Agrupamento e Agregações (Tabela 4)

PostgreSQL 14,2s a 14,5s | MySQL 472,7s a 478,3s

Este cenário apresenta a maior assimetria. O PostgreSQL conclui as queries em constantes 14 segundos, enquanto o MySQL extrapola para a casa dos 7 minutos.

O PostgreSQL utiliza um planejador baseado em custo (CBO). Se o conjunto estimado de chaves de agrupamento couber na RAM, o Postgres ativa o algoritmo HashAggregate. Mesmo nos cenários Sem Index ou com Índice Hash, o tempo se manteve em 14 segundos porque a criação de índices secundários em disco não altera a eficiência do HashAggregate interno em memória RAM, que processa os dados de forma massiva após o Sequential Scan.

O MySQL (InnoDB) exibe severas limitações arquiteturais ao processar agregações complexas. Historicamente, o otimizador do MySQL prioriza cargas de trabalho transacionais rápidos. Na ausência de um índice perfeitamente ajustado e ordenado cobrindo os campos de agrupamento e filtragem, o motor recorre ao algoritmo de Group-by com Ordenação Implícita.

MySQL 1,58GB} | PostgreSQL 1,74GB} a 1,83GB

O consumo do MySQL tem um componente de gerenciamento de tabelas de memória interna. Para resolver as cláusulas GROUP BY e HAVING sobre tabelas sem ordenação ideal em disco, o MySQL aloca RAM para consolidar as chaves de agregação. Estoura do limite máximo permitido pela variável, ao atingir esse limite, o motor congela a expansão de RAM da thread e delega o excedente a arquivos físicos temporários em disco.

O PostgreSQL diante de agregações massivas, o Otimizador Baseado em Custo (CBO) ativa o algoritmo HashAggregate. O Postgres calcula a quantidade de memória de trabalho necessária para criar extensas tabelas hash em memória RAM volátil. O SGBD prefere expandir agressivamente a pegada de memória do processo filho para acomodar os acumuladores matemáticos (COUNT, SUM, AVG), evitando o gargalo de escrita em disco e resolvendo a operação em escassos 14 segundos.

B) Filtragem Parcial Textual com Cláusula LIKE (Tabela 5)

PostgreSQL 16,7s a 17,0s | MySQL 28,9s a 29,0s

Neste cenário, avaliou-se o casamento de padrões de strings utilizando o caractere coringa (%). Ambos os SGBDs mantiveram tempos estáticos e lineares em todas as suas respectivas vertentes de indexação, demonstrando a invalidação completa de índices tradicionais. O posicionamento central ou inicial do caractere coringa (ex: LIKE '%database%' ou LIKE '%Silva') impede qualquer navegação hierárquica em estruturas indexadas padrão.

O PostgreSQL superou o MySQL em aproximadamente 1,7 vezes (16,9s contra 29,0s). Esse ganho decorre da eficiência do motor de armazenamento na leitura de páginas Heap sequenciais e do suporte nativo a Varreduras Sequenciais Paralelas. O Postgres divide o arquivo de dados em blocos e distribui a leitura e a checagem lógica das strings do LIKE entre múltiplos núcleos de CPU.

O MySQL/InnoDB, operando em threads de execução mais acopladas por query, processa a varredura linear de forma mais lenta, sofrendo também com o maior tamanho de página padrão do InnoDB (16KB) ao manipular colunas do tipo texto longo.

MySQL 815 a 839MB | PostgreSQL 1,25GB

O MySQL processa a query de forma linear através de uma única thread de execução, o consumo de RAM privada restringe-se ao espaço alocado para carregar os blocos textuais da tabela para análise e gerenciar o buffer de envio de resultados para o cliente. Teve um comportamento puramente transacional de leitura sequencial sem o uso de hashing auxiliar em memória.

O PostgreSQL teve um incremento de memória, reflexo direto da ativação do algoritmo de Varredura Sequencial Paralela. Para ler e avaliar as strings contidas nas colunas de texto longo, o processo principal recruta múltiplos processos trabalhadores. Cada worker gerencia seu próprio buffer de paginação e estado lógico de processamento de strings. O somatório das áreas de memória privada desses processos paralelos infla a estatística global de RAM da sessão, mas confere ao PostgreSQL uma velocidade de processamento maior.

C) Processamento de Junções Estruturais - JOINS (Tabela 6)

O cenário de junções exibe um equilíbrio competitivo notável entre os dois sistemas, com os tempos flutuando na janela de 56 a 64 segundos.

O PostgreSQL avalia três algoritmos principais de junção: Nested Loop, Merge Join e Hash Join. Nas consultas que cruzam tabelas de alta densidade sem predicados altamente seletivos, o Postgres prioriza o Hash Join.

O MySQL resolve junções através do algoritmo Nested Loop Join.

MySQL 1,58GB | PostgreSQL 1,73GB a 1,79GB

No MySQL quando a query realiza junções complexas sem o amparo de chaves primárias ou B+Trees ideais, o motor substitui o laço aninhado clássico pelo algoritmo de Hash Join. Para isso, ele consome RAM privada delimitada para criar tabelas hash temporárias em memória. Novamente, a volumetria excessiva fez o mecanismo colidir com a barreira máxima do alocador TempTable, forçando a estabilização do consumo no teto.

No PostgreSQL quando há índices disponíveis nas chaves estrangeiras, o otimizador frequentemente combina algoritmos de Nested Loop com Index Scan ou Merge Join (que exige

ordenação prévia das fatias). Para coordenar a fusão de múltiplos fluxos indexados vindos de tabelas concorrentes, o motor expande dinamicamente a alocação de buffers de classificação em memória RAM.

D) Resolução de Subconsultas Aninhadas (Tabela 7)

PostgreSQL 0,77s a 1,18s | MySQL 8,73s a 10,08s

No processamento de subconsultas lógicas, o PostgreSQL demonstrou uma superioridade de até 10 vezes em relação ao MySQL.

O PostgreSQL possui um otimizador extensivo que aplica regras de Desaninhamento antes de computar o plano físico. Ao analisar operadores como IN ou EXISTS, o Postgres transforma algoritmicamente a subconsulta em uma estrutura equivalente de Semi-Join ou Anti-Join. Uma vez convertida em Semi-Join, a query passa a ser tratada com a mesma eficiência de um Hash Join plano em memória RAM. Nos cenários indexados por B+Tree ou Hash, o motor utiliza os índices internos de forma atômica para validar os conjuntos rapidamente. No cenário Sem Index, o tempo sobe ligeiramente, provando que a reescrita lógica mitiga a ausência de caminhos físicos auxiliados.

O MySQL apresenta dificuldades estruturais com subconsultas complexas, especialmente aquelas que envolvem agregações ou correlações diretas. Para subconsultas independentes (operador IN), o MySQL cria uma tabela temporária interna em memória para armazenar o resultado do bloco aninhado e, posteriormente, indexá-la dinamicamente. Se o volume de dados extrapola os limites de memória de trabalho, essa tabela é jogada para o disco. Em subconsultas correlacionadas (EXISTS), o motor muitas vezes falha em converter a instrução em uma junção linear, sendo forçado a executar o bloco interno como um laço repetitivo para *cada* linha avaliada no bloco externo. Isso faz com que o tempo de processamento permaneça estagnado independentemente de haver índices Hash ou B+Tree ativos, visto que o gargalo reside no número massivo de iterações síncronas de CPU necessárias para consolidar o resultado final.

MySQL 839 MB | PostgreSQL 1,07GB a 1,32GB

O MySQL diante de operadores como IN e NOT IN, o motor tende a isolar a subconsulta independente e realizar a Materialização, gravando o resultado do bloco aninhado em uma tabela temporária interna em memória e gerando um índice automático sobre ela. Seguiu um plano de execução rígido e padronizado por seu otimizador, alocando um bloco de RAM estático para a tabela materializada interna.

O PostgreSQL exibiu uma oscilação altamente correlacionada à presença de caminhos físicos: *Cenários Indexados (B+ e Hash 1,07 a 1,19 GB)* - quando há índices nas tabelas internas, o Postgres consegue resolver os predicados aninhados rapidamente por acessos diretos à árvore, poupando a necessidade de grandes buffers privados. *Cenário Sem Índice (1.387.934,0 KB)* - na ausência total de índices, o Otimizador do PostgreSQL aplica transformações algébricas de Desaninhamento, convertendo as subconsultas em Hash Semi-Joins. Para executar o Semi-Join sem o apoio de índices em disco, o motor aloca uma quantidade massiva de RAM privada para erguer tabelas hash temporárias em tempo de execução. Esse comportamento eleva o consumo de RAM, mas permite que o PostgreSQL resolva a subconsulta rapidamente.

4. Conclusões

A realização deste estudo experimental permitiu mapear a eficiência algorítmica entre o PostgreSQL e o MySQL. Os resultados expõem que cada SGBD responde de forma radicalmente distinta dependendo do perfil da carga de trabalho (transacional / analítico), da volumetria dos dados manipulados e da presença ou ausência de estruturas de indexação auxiliares.

As operações de modificação de dados expuseram o contraste mais profundo entre os modelos de concorrência e controle de isolamento das ferramentas. O PostgreSQL demonstrou uma agilidade avassaladora e consistente, concluindo inserções, deleções e atualizações pontuais por ID. O MySQL, embora rápido no INSERT, sofreu uma severa degradação transacional no DELETE e no UPDATE. Esse colapso de tempo no MySQL reflete a penalidade síncrona provocada por varreduras completas (Full Table Scans) consecutivas e pelo esforço de reordenação física de páginas do InnoDB quando chaves e restrições ideais não estão explicitamente ativas em tabelas clonadas. No PostgreSQL, a arquitetura append-only do MVCC posterga o custo de limpeza para processos assíncronos de fundo (Vacuuming), garantindo tempos de resposta instantâneos na camada de aplicação. O modelo multiprocesso do PostgreSQL provou ser altamente enxuto para DMLs puros, uma vez que as atualizações ocorrem diretamente nas páginas Heap e nos buffers globais. O MySQL, sob o modelo Thread-Per-Connection, manteve uma pegada inflada pelo overhead de estruturas de sessão obrigatórias da thread do InnoDB.

As buscas simples evidenciaram os limites teóricos e a sensibilidade de cada SGBD perante a indexação secundária e o acesso físico ao disco. Nas buscas por Chave Primária, o MySQL obteve uma vitória marginal. Esse ganho é justificado pela arquitetura de Tabelas Organizadas por Índice (IOT) do InnoDB: como o dado físico está acoplado ao nó folha da árvore Clustered, o MySQL localiza a linha em um único passo. Contudo, nas Chaves Não-Primárias, o cenário inverteu-se drasticamente: o PostgreSQL manteve resiliência recorrendo ao algoritmo Sequential Scan ou Bitmap Index Scan lineares, enquanto o MySQL subiu na ausência de índices e só conseguiu melhorar quando amparado por uma Árvore B+ ideal, sofrendo com o alto custo de múltiplos Key Lookups. Em acessos pontuais (Chave Primária), manteve-se o comportamento estável de uso de memória. Porém, ao varrer atributos secundários de larga escala (Chaves Não-Primárias), ambos os motores expandiram o consumo agressivamente para a casa dos gigabytes. O MySQL estabilizou o consumo pelo gerenciador de memória TempTable, despejando o excesso de linhas processadas em arquivos de paginação física em disco. O PostgreSQL demonstrou maior elasticidade, alocando buffers de página privados em RAM para garantir que a varredura e o fluxo de blocos lidos fossem mantidos inteiramente na memória volátil de alta velocidade.

As operações de busca complexas consolidaram as maiores assimetrias de engenharia de software documentadas no projeto:

- Agrupamentos e Agregações (GROUP BY / HAVING): O PostgreSQL superou o MySQL. O PostgreSQL alcançou este feito utilizando o algoritmo HashAggregate, que cria tabelas hash dinâmicas na memória RAM privada para acumular as métricas lógicas na velocidade da CPU. O MySQL, impedido de expandir sua RAM além do teto do TempTable, foi sufocado pelo algoritmo de ordenação implícita com passagem externa em disco, gerando um severo gargalo de escrita de arquivos temporários.
- Subconsultas Aninhadas (IN, EXISTS, NOT IN): O PostgreSQL foi até 10 vezes mais veloz que o MySQL. O otimizador do PostgreSQL aplica regras matemáticas avançadas de Desaninhamento (Subquery Unnesting), convertendo as queries aninhadas em estruturas lineares de Hash Semi-Joins ou Anti-Joins. Para isso, o Postgres expande a RAM quando não há índices em disco, enquanto o MySQL fica preso a planos engessados de Materialização e laços repetitivos síncronos de CPU por linha.

- Casamento de Padrões (Cláusula LIKE) e Junções (JOINS): No processamento de strings parciais com caracteres coringa (%), ambos os SGBDs comprovaram a invalidação teórica de índices baseados em Hashing e B+Tree à esquerda, resultando em leituras de tabelas completas. No entanto, o suporte a Varreduras Sequenciais Paralelas deu vantagem ao PostgreSQL, sacrificando mais memória RAM devido ao overhead de gerenciar múltiplos processos trabalhadores concorrentes. Nas Junções, houve equilíbrio competitivo, expondo que o motor moderno do MySQL consegue reagir eficientemente via Hash Join em memória através do buffer de junção quando índices secundários ineficientes em chaves estrangeiras não o induzem ao erro de realizar loops aninhados com saltos aleatórios no disco.

O PostgreSQL provou possuir um Otimizador Baseado em Custo (CBO) substancialmente mais maduro e inteligente para operações de BI, relatórios e processamento de conjuntos. Sua capacidade de alocar memória RAM de forma elástica, dinâmica e paralela (Hash Aggregate, Hash Join, Bitmap Scan, Parallel Workers) compensa a ausência de índices e esmaga o MySQL em tempos de resposta em qualquer cenário analítico complexo, ao custo de uma exigência maior de hardware (memória volátil).

O MySQL brilha em cenários de alta seletividade baseados estritamente na Chave Primária. Sua arquitetura de índice Clustered oferece latências de leitura imbatíveis para acessos pontuais por ID. No entanto, o MySQL demonstra ser um sistema extremamente engessado e sensível: a ausência de índices secundários perfeitamente calibrados ou o uso de queries mal modeladas penaliza o motor de forma dramática, expondo restrições rígidas de gerenciamento de buffers que travam o consumo de RAM e transferem a carga de processamento para operações lentas em disco.

O estudo comprova de forma definitiva que a simples criação de índices sem planejamento não garante performance. Índices não-ideais ou estruturas auxiliares submetidas a filtros textuais centrais adicionam custos severos de armazenamento e manutenção na RAM sem trazer qualquer benefício de velocidade, reforçando que a eficiência de um banco de dados reside no desenho cirúrgico e no conhecimento íntimo dos algoritmos internos do SGBD escolhido.

5. Link para o Repositório com o Código dos Experimentos

O código-fonte completo contendo os scripts de criação de tabelas, inserção de dados, automação de testes em Python e as planilhas brutas de medições estão disponíveis em:

<https://github.com/SQL-2026-Trabalho/ep1>

6. Link para o Vídeo de Apresentação do Trabalho

A gravação da apresentação dos resultados pode ser acessada pelo link:

****inserir link do youtube

7. Referências

- [1] SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 7. ed. Rio de Janeiro: GEN LTC, 2020.
- [2] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 15.0 Documentation*. Disponível em: <https://www.postgresql.org/docs/15/index.html>
- [3] ORACLE CORPORATION. *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson Education, 2018.
- [5] GARCIA-MOLINA, Hector; ULLMAN, Jeffrey D.; WIDOM, Jennifer. *Database Systems: The Complete Book*. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2008.
- [6] PETROV, Alex. *Database Internals: A Deep Dive into How Distributed Data Systems Store, Treat and Organize Data*. Sebastopol: O'Reilly Media, 2019.
- [7] KLEPPMANN, Martin. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol: O'Reilly Media, 2017.
- [8] LIGHTSTONE, Sam; TEOREY, Toby; NADEAU, Tom. *Physical Database Design: the database professional's guide to exploiting indexes, views, storage, and more*. San Francisco: Morgan Kaufmann, 2007.
- [9] COGMAN, Craig. *PostgreSQL Query Optimization: The Ultimate Guide to Building Efficient Queries*. Berkeley: Apress, 2021.